DigitalDepot

Fejlesztői Dokumentáció: 2
Backend függőségek: 2
Frontend függőségek: 2
.env változók 3
Adatbázis Felépítése: 3
categories 4
coupons 4
messages 4
Mappaszerkezet 8
API végpontok 9
Kép feltöltésének folyamata: 13
Rendelés leadása: 15
Autentikáció 19
apiFetch() 20
Socket.io 22
frontend architektúra 24
Tesztek 25
Továbbfejlesztési javaslatok 25
Források 26

Fejlesztői Dokumentáció:

Fejlesztők: Dékán Donát, Nagy Dávid
Használt nyelvek: Node.js, React.js, JavaScript, MySql

Backend függőségek:

- bcrypt -> jelszvak titkosítása
- cookie-parser -> a kérés fejlécében küldött cookie feldologzása
- cors -> cross-origin kérések engedélyezése
- crypto -> random kód generálásához és randomizáció nélüli hasheléshez
- dotenv -> a lokális változók beolvasásához
- express -> az express keretrendszer használatához
- express-rate-limit -> a kérések számának korlátozása
- fs -> kép fájlok törléséhez
- helmet -> security fejlécek beállítása
- jsonwebtoken -> a JWT használatához
- multer -> formdata feldolgozása
- mysql2 -> kapcsolat létrehozása az adatbázissal
- nodemailer -> email küldése
- sharp -> kép formázása/validálása
- socket.io -> websocket kapcsolathoz

Frontend függőségek:

- jwt-decode -> JWT olvasására
- react, react-dom, react-router-dom -> React.js könyvtár
- socket.io-client -> kliens oldali websocket

.env változók

IP = backend futtatási ip-je

PORT = backend port-ja

HOST = adatbázis host-ja

USER = adatbázishoz használt user

PASSWORD = az adatbázis felhasználóhoz tartozó jelszó (ha nncs, akkor üres marad)

DATABASE = az adatbázis neve

SECRET = egy titkos string, amivel a JWT signature jön létre

EMAIL_PASSWORD = az email cím applikációs kulcsa

EMAIL_USER = az email cím

Adatbázis Felépítése:

users:

- userId – int auto-increment elsődleges kulcs
- userName – varchar(255) felhasználónév, ismétlődhet
- hashedPassword – jelszó, bcryptel hashelt
- email – email cím, UNIQUE
- role – varchar(255) lehetséges: ’user’, ’admin’, ’owner’
- bankAccountNumber – varchar(50), 24 karakter(szerver ellenőrzi)
- chatTopic – varchar(50), lehetséges: ’szállítás’, ’fizetés’, ’termékhiba’, ’garancia’, ’visszaküldés’, ’rendelés’, ’egyéb’

products:

- prodId – int auto-increment elsődleges kulcs
- productName – varchar(255) termék neve
- productDescription – varchar(255) termék rövid leírása
- productPrice – double termék ára
- productImg – varchar(255) termék képének relatív elérési útvonala
- categoryId – a termék kategóriájának azonosítója IDEGEN KULCS(categories.categoryId)
- averageRating – decimal, az értékelések átlaga (trigger számolja)
- reviewCount – int, értékelések száma
- conditionState – varchar(255) termék állapota, alapértelmezett NULL
- quantity – int a leltáron lévő termék mennyiség száma

carts:

- userId – int felhasználó azonosítója IDEGEN KULCS(users.userId)
- productId – int kosárban lévő termék azonosítója IDEGEN KULCS(products.prodId)
- quantity – int kosárban lévő termékek darabszáma

categories:

- categoryId – int elsődleges kulcs kategória azonostója
- categoryName – varchar(255) kategória neve
- parentId – int lehetséges: másik kategória azonosítója alkategória esetén, null fő kategóri esetén

coupons:

- id – int elsődleges kulcs auto-increment
- code – varchar(255) a kuponkód
- value – int a kupon értéke
- usedAt – date lehetséges: null ha nincs használva, a felhasználás dátuma
- orderId – int a rendelésazonosítója amin használva lett

messages:

- id – unsigned bigint auto-increment, elsődleges kulcs
- sender – int, a küldő felhasználó azonosítója, IDEGEN KULCS (users.userId)
- message – text, az üzenet tartalma
- sentAt – datetime, az üzenet küldésének időpontja
- recipientId – int, a címzett felhasználó azonosítója (lehet NULL, ha adminnak megy)
- unread – tinyint(1), alapértelmezett értéke 1 (olvasatlan), jelzi az üzenet állapotát

message_logs:

- id – unsigned bigint, a törölt üzenet eredeti azonosítója
- sender – int, a küldő azonosítója
- message – text, az üzenet szövege
- sentAt – date, a küldés dátuma
- recipientId – int, a címzett azonosítója
- unread – tinyint(4), az üzenet állapota a törléskor
- *Megjegyzés: A táblát a log_messages trigger tölti fel üzenet törlésekor.*

orders:

- orderId – int auto-increment, elsődleges kulcs
- userId – int, a rendelést leadó felhasználó azonosítója, IDEGEN KULCS (users.userId)
- totalAmount – int, a rendelés végösszege
- shippingAddress – varchar(255), a szállítási cím
- orderDate – datetime, a rendelés leadásának ideje (alapértelmezetten a rendszeridő)
- paymentMethod – varchar(50), a fizetés módja (pl. 'utanvet')
- couponCode – varchar(50), a felhasznált kupon kódja
- status – varchar(50), a rendelés állapota (alapértelmezett: 'Függőben')

order_items:

- orderId – int, a rendelés azonosítója, ÖSSZETETT ELSŐDLEGES KULCS RÉSZE, IDEGEN KULCS (orders.orderId)
- productId – int, a termék azonosítója, ÖSSZETETT ELSŐDLEGES KULCS RÉSZE, IDEGEN KULCS (products.prodId)
- quantity – int, a rendelt mennyiség
- price – int, a termék egységára a rendelés pillanatában

reviews:

- reviewId – int auto-increment, elsődleges kulcs
- userId – int, az értékelő felhasználó azonosítója, IDEGEN KULCS (users.userId)
- productId – int, az értékelt termék azonosítója, IDEGEN KULCS (products.prodId)
- rating – int, az értékelés számszerű értéke (1 és 5 között)
- comment – text, szöveges vélemény
- reviewDate – datetime, az értékelés időpontja
- *Megjegyzés: Új értékelés beszúrásakor a recalculate_rating_after_insert trigger frissíti a termék átlagértékelését.*

used_product_submissions:

- submissionId – int auto-increment, elsődleges kulcs
- userId – int, a beküldő felhasználó azonosítója, IDEGEN KULCS (users.userId)
- productName – varchar(255), a használt termék neve
- productDescription – text, a termék leírása
- conditionState – varchar(255), a termék állapota
- productImage – varchar(255), a termékről készült kép útvonala
- status – varchar(255), a beküldés állapota (pl. 'pending', 'listed')
- submissionDate – datetime, a beküldés dátuma
- adminOfferPrice – int, az admin által ajánlott felvásárlási ár

refreshtokens:

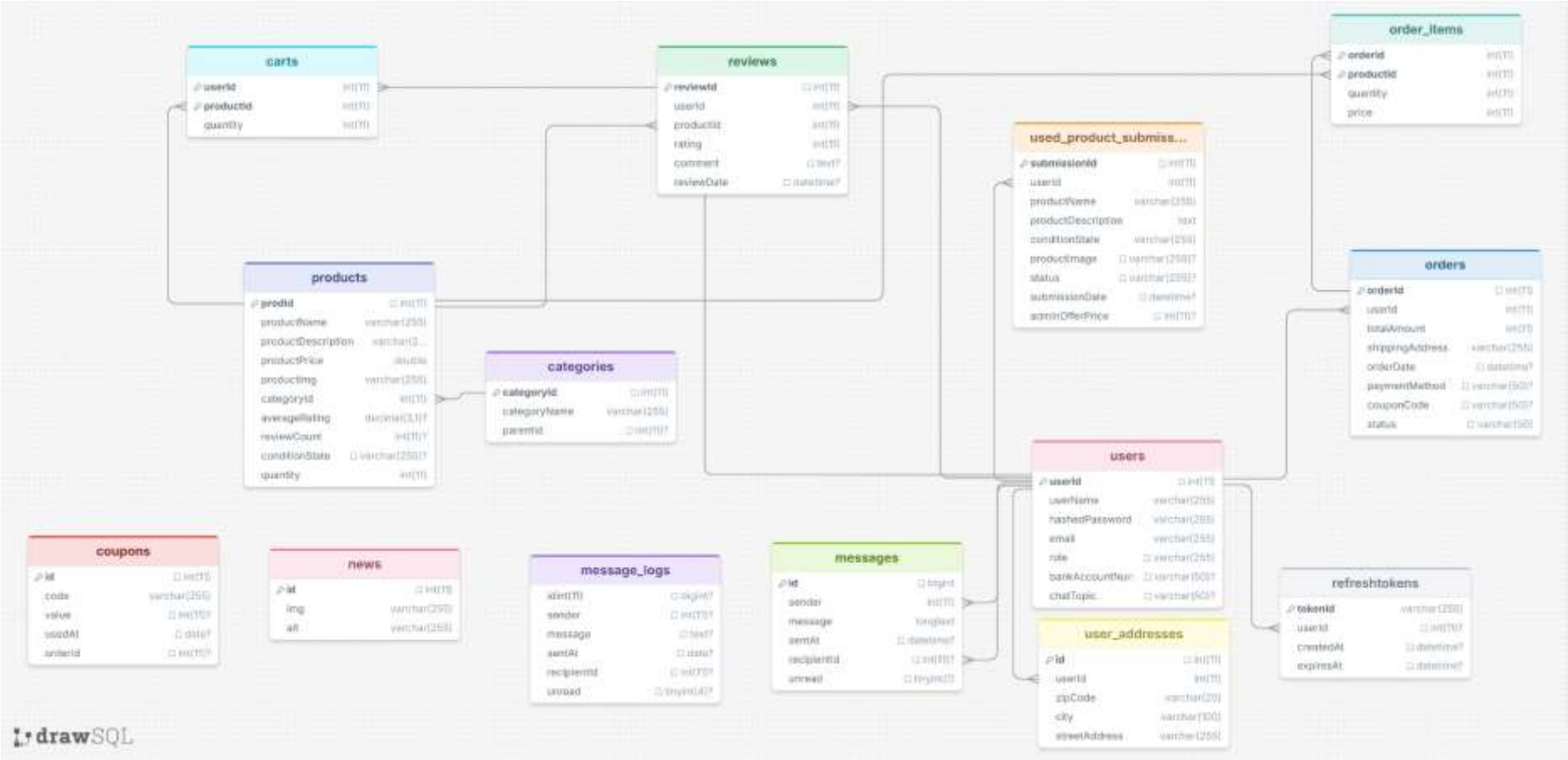
- tokenId – varchar(255), elsődleges kulcs, a token azonosítója
- userId – int, a tokenhez tartozó felhasználó, IDEGEN KULCS (users.userId)
- createdAt – datetime, a token generálásának ideje
- expiresAt – datetime, a token lejárati ideje

user_addresses:

- id – int auto-increment, elsődleges kulcs
- userId – int, a felhasználó azonosítója, IDEGEN KULCS (users.userId)
- zipCode – varchar(20), irányítószám
- city – varchar(100), város
- streetAddress – varchar(255), utca és házszám

news:

- id – int auto-increment, elsődleges kulcs
- img – varchar(255), a hírhez vagy bannerhez tartozó kép útvonala
- alt – varchar(255), a kép helyettesítő szövege


carts
reviews
order_items
used_product_submiss...
products
categories
orders
users
coupons
news
message_logs
messages
user_addresses
refreshtokens
drawSQL

Mappaszerkezet

```
backend
├── __tests__
├── models
├── routes
├── uploads
│   ├── news
│   └── products
├── server.js
└── .env
frontend
├── __tests__
├── src
│   ├── assets
│   │   ├── NavImages
│   │   ├── sliderImg
│   │   └── util
│   ├── components
│   ├── data
│   ├── app.jsx
│   ├── index.jsx
│   └── main.css
└── index.html
```

API végpontok

/user

Végpont	Leírás	Jogosultság
POST /register	Új felhasználó létrehozása	Bárki
POST /login	felhasználó bejelentkeztetése, tokenek kiosztása	Bárki
POST /logout	kijelentkezés, refresht token törlése	Bárki
GET /refresh	új access token kérése	Bárki (refresh token)
POST /reset-code	helyreállító kód kérése	Bárki
POST /reset-password	jelszó visszaállítása kóddal	Bárki
GET /profile	visszaadja a felhasználó profil adatait	Bejelentkezett
PATCH /bank-account	beállítja a felhasználó számláját (HU 24számjegy)	Bejelentkezett
PATCH /chat-topic	Az aktuális chat téme módosítása	Bejelentkezett
GET /adresses	Visszaadja a felhasználó mentett címeit	Bejelentkezett
POST /adresses	Új cím mentése	Bejelentkezett
DELETE /adresses/:id	Törli az adott címet	Bejelentkezett
GET /messages	Visszaadja a felhasználó chat beszélgetését	Bejelentkezett
POST /readmessages	megjelöli olvasottként az üzeneteket	Bejelentkezett

/products

Végpont	Leírás	Jogosultság
GET /	visszaadja a termékeket	Bárki
GET /:productId	visszaadja az adott termék adatait	Bárki
GET /search/:string	név és leírás alapú keresés	Bárki
GET /category/:categoryId	az adott kategória termékeit adja vissza	Bárki

/cart

Végpont	Leírás	Jogosultság
GET /	a felhasználó kosarának tartalma és végösszege visszaadása	Bejelentkezett
POST /add/:id/:quantity	az adott terméket hozzáadja a felhasználó kosarához	Bejelentkezett

/order

Végpont	Leírás	Jogosultság
GET /items/:orderId	Az adott rendelésben lévő termékeket adja vissza	Bárki
GET /my-orders	A felhasználó korábbi rendeléseit adja vissza	Bejelentkezett
POST /place-order	A felhasználó kosarában lévő termékek megrendelése	Bejelentkezett

/reviews

Végpont	Leírás	Jogosultság
GET /:productId	Az adott termék értékeléseit adja vissza	Bárki
GET /my-review/:productId	A felhasználó saját értékelésének lekérése az adott termékre	Bejelentkezett
POST /	Az adott termékre írt értékelés közzététele	Bejelentkezett
PATCH /:reviewId	Egy már közzétett értékelés módosítása	Bejelentkezett
DELETE /:reviewId	Az adott értékelés törlése	Bejelentkezett

/news:

Végpont	Leírás	Jogosultság
GET /	A jelenlegi hírek lekérése	Bárki

/coupon:

Végpont	Leírás	Jogosultság
POST /check	Ellenőrzi a kupon létezését és visszaadja az értékét ha létezik	Bárki

/categories

Végpont	Leírás	Jogosultság
GET /	visszaadja a kategóriák listáját	Bárki
GET /:categoryId	Egy adott kategória termékeit adja vissza	Bárki

/adminRoute

Végpont	Leírás	Jogosultság
GET /statistics	visszaadja a webshop statisztikáit	Owner
POST /register	új admin fiók regisztrálása	Owner
GET /users	visszaadja az összes jelenlegi felhasználót	Owner
PATCH /users/:userId/role	módosítja az adott felhasználó szerepkörét	Owner
DELETE /users/:userId	törli az adott felhasználót és a hozzá tartozó adatokat	Owner
GET /orders	visszaadja az összes korábbi rendelést	Admin
GET /orders/:orderId	visszaadja az adott rendelés adatait	Admin
DELETE /orders/:orderId	törli az adott rendelést	Admin
PATCH /orders/:orderId/status	módosítja a rendelés állapotát	Admin
PATCH /products/addInventory	hozzáad az adott termék mennyiségéhez	Admin
PATCH /products/:prodId	módosítja a termék paramétereit	Admin
GET /messages/:id	Az adott felhasználóval történő beszélgetés lekérése	Admin
Végpont	**Leírás**	**Jogosultság**
GET /messages	az összes beszélgetés lekérése	Admin
DELETE /messages/:id	Az adott felhasználó beszélgetésének törlése(message_logs táblába kerül)	Admin
POST /addProduct	új termék létrehozása	Admin
PATCH /product/:productId	módosítja a termék képét	Admin
POST /news	új hír hozzáadása	Admin
DELETE /news/:id	az adott hír törlése	Admin
POST /coupon	kupon hozzáadása	Admin

/used-products

Végpont	Leírás	Jogosultság
POST /submit	listázza az eladni kívánt terméket ajánlat kérésre	Bejelentkezett, Számlaszám
GET /my-submissions	Visszaadja a leadott használt	Bejelentkezett
GET /admin/all	Visszaadja az összes felhasználó által beküldött terméket	Admin
PATCH /admin/status	ajánlatot tesz a felhasználónak	Admin
PATCH /user-response	elfogadja/elutasítja az admin által tett ajánlatot	Bejelentkezett
POST /admin/list-product	listázza eladásra az adott terméket	Admin

Kép feltöltésének folyamata:

Termék -> /uploads/products

Hír -> /uploads/news

A multer szűri a fájltípust, a megengedettek a következők: .jpg, .jpeg, .png

```
const allowedExtensions = /jpeg|jpg|png/;


  const extName = allowedExtensions.test(
    path.extname(file.originalname).toLowerCase()
  );


  const mimeType = allowedExtensions.test(file.mimetype);
```

Ha a kiterjesztés nem felel meg ennek a szűrőnek, akkor egy hibát dob, amit az upload.js kap el és küldi el a felhasználónak

```
if (extName && mimeType) {
    return cb(null, true);
  } else {
    cb(new Error('Csak .jpg, .jpeg és .png fájlok engedélyezettek!'), false);
  }
```

A validateImage.js ellenőrzi, hogy valóban kép került-e feltöltésre és nem csak módosították-e a kiterjesztést. Ezt a sharp csomag segítségével ellenőrizzük, ami hibát dob, ha nem képet adunk meg neki. Ezesetben a fájl törlésre kerül

```
await sharp(req.file.path).metadata();
```

az imageProcessor.js ezután a paraméterként megadott méretekre alakítja a képet.

termékkép: 800*600px

hírKép: 1200*600px

A méretarányokat megtartja, a kimaradó éleket pedig fehér háttérrel látja el. Ezután a feldolgozatlan képet felülírja az átméretezettel.

```
await sharp(originalPath)
      .resize(width, height, {
        fit: 'contain',
        background: { r: 255, g: 255, b: 255, alpha: 1 },
      })
      .toFile(tempPath);


    await fs.rename(tempPath, originalPath);
```

Rendelés leadása:

Végpont

A rendelés leadása a POST /order/place-order végpontal történik meg. Paraméterként vár egy szállítási címet, egy fizetési módot, és egy opcionális kupon kódot, majd meghívja az Order.Save() metódust.

Order.save()

Az adatintegritás érdekében egy adatbázis tranzakcióval biztosítjuk, hogy ne legyen félkész rendelés leadva.

```
connection = await db.getConnection();

await connection.beginTransaction();
```

A kosárban lévő termékeket lekéri az adatbázis, a készletükkel együt, amit írásra zárol commit vagy rollback hívásáig, hogy megakadájozza a racecondition-t.

```
const [cartItems] = await connection.execute(

    `SELECT products.prodId, products.productPrice, products.quantity AS stock, carts.quantity

     FROM products

     INNER JOIN carts ON products.prodId = carts.productId

     WHERE carts.userId = ? FOR UPDATE;`,

    [this.userId]

  );
```

Üres kosár esetén sikertelen lesz a kérés.

Lekéri a kosárban lévő termékeket, majd ellenőrzi, hogy van-e belőlük készleten. Ha nem, akkor sikertelen a kérés és rollback-el.

```
if (cartItems.length === 0) {
    await connection.rollback();
    return { result: 'fail', message: 'Üres a kosár' };
  }

  for (const product of cartItems) {
    if (product.stock < product.quantity) {
      await connection.rollback();
      return {
        result: 'fail',
        message: 'Egy vagy több termék nincs készleten',
      };
    }
  }
```

Összeadja a termékek árának összegét, majd ellenőrzi, hogy van-e érvényes kuponkód, és levonja az értékét a végösszegből. Ha a kedvezmény értéke nagyobb a végeredménynél, akkor a végösszeg 0 lesz.

```
let totalAmount = 0;
  cartItems.forEach((item) => {
    totalAmount += item.productPrice * item.quantity;
  });
  let discount;
  if (this.couponCode != '') {
    couponResult = await Coupon.check(this.couponCode);
    if (couponResult.result == 'success') {
      totalAmount = totalAmount - couponResult.data.value;
      discount = couponResult.data.value;
      if (totalAmount < 0) {
        totalAmount = 0;
      }
    }}
```

Ezután beilleszti a rendelés adatait az orders táblába

```
const [orderResult] = await connection.execute(
      `INSERT INTO orders (userId, totalAmount, shippingAddress, paymentMethod, couponCode, orderDate)
       VALUES (?, ?, ?, ?, ?, NOW())`,
      [
        this.userId,
        totalAmount,
        this.shippingAddress,
        this.paymentMethod,
        this.couponCode,
      ]
    );
```

Ezután az order_items táblába teszi a rendelésben szereplő termékeket, majd csökkenti a leltárban lévő mennyiéget.

```
const orderId = orderResult.insertId;


    for (let item of cartItems) {
      await connection.execute(
        `INSERT INTO order_items (orderId, productId, quantity, price)
         VALUES (?, ?, ?, ?)`,
        [orderId, item.prodId, item.quantity, item.productPrice]
      );


      await connection.execute(
        `UPDATE products SET quantity = quantity - ? WHERE prodId = ?`,
        [item.quantity, item.prodId]
      );
    }
```

Ha sikeres volt a kupon felhasználása, akkor az adatbázisban módostítódik a kupon állapota

```
if (couponResult && couponResult.result == 'success') {
    await connection.execute(
      'UPDATE coupons SET coupons.usedAt = NOW(), coupons.orderId=? WHERE coupons.code = ?;',
      [orderId, this.couponCode]
    );
  }
```

Ezután nodemailer segítségével kiküldi a rendelést visszaigazoló emailt és üríti az user kosarát.

Mindezek után commitolja a változtatásokat, és egy sikeres választ ad vissza

```
  await connection.commit();
  connection.release();
  return { result: 'success' };
```

Autentikáció

Tokenek

Bejelentkezéskor a felhasználó a JSON válaszban kap egy access tokent (JWT) és htpOnly cookieban egy refresh tokent(string).
A JWT token tartalmazza a felhasználó azonosítóját, illetve szerepkörét. A token 15 percig érvényes, a tárolási helye a localstorage ’token’ néven.

A refresh token egy véletlenszerű 128 karakteres string, amely crypto csomag által sha256-os hasheléssel lesz eltárolva az adatbázisba. A token bekerül a httpOnly cookieba, ami csak a /user/refresh API hívásakor kerül elküldésre.

tokenVerify

tokenwerify middleware segít ellenőrizni az accesstoken érvényességét és meghatározni a felhasználó kilétét.

A fejlécből kiszedi a tokent, majd ellenőrzi, hogy nem-e lett módosítva, nem-e járt le. Hiányzó token esetén 401-es hibát küld vissza. Sikeres ellenőrzés esetén csatolja arequest.user-be a token tartalmát és továbbengedi a kérést.

```
const decodedToken = await verifyAsync(token, process.env.SECRET);

req.user = decodedToken;

next();
```

Ha az ellenőrzés sikertelen, akkor 401-es hibát ad vissza

verifyAdmin

a verifyAdmin middleware működése megegyezik a tokenVerify-éval, azzal a különbséggel, hogy a verifyAdmin ellenőrzi a felhasználó szerepkörét. user esetén 403-as hibát ad vissza.

```
if (decodedToken.role == 'admin' || decodedToken.role == 'owner') {

  req.user = decodedToken;

  next();

} else {

  return res

    .status(403)

    .json({ result: 'fail', message: 'no authorization!' });

}
```

Token frissítés

Lejárt vagy elveszett access token esetén a /user/refresh API meghívásával igényelhető új. A végpont kiolvassa a cookieból a tokent és kikeresi a hozzá tartozó adatokat az adatbázisból.Amennyiben ez nem sikerül, úgy hibás kérésként fog visszatérni.

sikeres felhasználó beazonosítás esetén létrehoz egy új access tokent, törli a régi refresh tokent az adatbázisból, majd egy újat hoz létre, amit beilleszt a cookieba.

apiFetch()

működése

az apifetch egy frontend segédfüggvény, amely lehetővé teszi az autentikáció zökkenőmentes működését.

paraméterként egy API útvonalát kéri, és egy objektumot, amely a fetch beállításait kell tartalmazza. az utóbbi elhagyásakor alapértelmezetten GET metódust állít be fejlécek és body nélkül.
amennyiben a localsotrageben megtalálja az accesstokent, úgy azt automatikusan csatolja a fejlécek közé.

Az apiFetch automatikusan ellenőrzi, hogy FormData vagy JSON típusú adatot próbálunk-e csatolni a body-ba, majd ennek megfelelően állítja be a fejlécet erről.

```
const isFormData = options.body instanceof FormData;

if (!isFormData) {

    settings.headers['Content-Type'] = 'application/json';

  }


  if (options.body !== undefined) {

    settings.body = isFormData ? options.body : JSON.stringify(options.body);

  }
```

Ezután meghívja a paraméterként megadott végpontot.

Sikeres kérés esetén a visszaadja a választ, 401-es hiba esetén pedig meghívja a /user/refresh végpontot új token igénylésért, majd az új tokennel újrapróbálja az eredei kérést. Ha ez sem sikeres, vagy 401 helyett más hibát kap, akkor megpróbálja kiolvasni aválasz message attribútumát, ellenkező esetben pedig egy előre definiált üzenetet ad meg, majd hibaként visszadobja.

```
    let errorData;
    try {
      errorData = await response.json();
    } catch (err) {
      errorData = { message: response.statusText };
    }
    const error = new Error(errorData.message || 'Hiba történt');
    error.status = response.status;
    error.data = errorData;
    throw error;
```

Példa használat

```
try {
  const data = await apiFetch('/products', { method: 'GET' });
  console.log(data);
} catch (err) {
  console.error(err.message);
}
```

Socket.io

A support chat a socket.io segítségével működik.

A chat komponens betöltésekor a JWT token alapján azonosítja be a felhasználót, majd a socket.user-be teszi a tartalmát.
Admin és Owner esetén a room_admin szobába kerülnek, user esetén a room_userId szobába.

```
if (socket.user.role === 'admin' || socket.user.role === 'owner') {
        socket.join('room_admin');
      } else {
        socket.join(`room_${socket.user.id}`);
      }
```

A send_message esemény a szerepkör alapján küldi ki az üzeneteket.

user esetén a következőképpen néz ki:

```
const message = {
          text: messageText,
          sender: id,
          date: Date.now(),
        };
```

admin és owner pedig így:

```
const message = {
            text: messageText,
            sender: id,
            date: Date.now(),
            recipientId: data.recipientId,
          };
```

ez alapján a recipientId hiánya/jelenléte segítségével könnyen eldönthető, hogy ki küldte az üzenetet. Ezután receive_message eseményként kiküldi a jogosult felhasználóknak az üzenetet és menti adatbázisba azt.

Az adminisztrátori chat oldalon a receive_message eseménynél ellenőrzi, hogy az adott user megtalálható-e már a betöltött üzenetek közt. Ha ott van, akkor az adott felhasználó üzenetei közé teszi azt be, ellenkező esetben pedig lekéri az adatait az adatbázisból,és új objektumot hoz létre hozzá.

A felhasználónál az üzenetek egy egyszerű listában vannak csak eltárolva, hiszen ott nem kell több felhasználó közt váltogatni.

Ha a felhasználó bezárná az oldalt, vagy hiba miatt megszakadna a kapcsolat, akkor a chat legközelebbi megnyitásakor lekéri az üzenet előzményeit, így nem veszik el semmi.

frontend architektúra

React Router

Általános útvonalak:

- /: Főoldal, a termékgaléria és a terméklista megjelenítése.
- /category/:categoryId: Szűrt terméklista egy adott kategória alapján.
- /search: Keresési eredmények megjelenítése query paraméter (?q=...) alapján.
- /product/:productId: Egy konkrét termék részletes adatlapja.

Védett útvonalak:

- Felhasználói: /profile (személyes adatok), /my-orders (rendelési előzmények), /used-products (használt termék beszámítás), /checkout (fizetési folyamat).
- Adminisztrátori: /admin (statisztikák, készletkezelés), /adminChat (ügyfélszolgálati felület).

Globális Statek

- newsData: A főoldali galériában megjelenő hírek és képek listája.
- isLoginOpen: A bejelentkező modális ablak láthatóságát vezérli.
- isCartOpen: Az oldalsó kosár panel nyitott/zárt állapotát jelzi.
- isProfileOpen: A profilbeállítások (gyorsmenü) megjelenítéséért felel.
- isMobileMenuOpen: Mobilnézetben a kategóriaválasztó menü állapotát tárolja.
- isMobileMenuClosing / isCartClosing: Az animált bezáráshoz használt segédállapotok.
- refreshTrigger: Egy számláló, aminek növelésével kényszeríthető a profiladatok újratöltése a komponensekben.
- isChatOpen: A lebegő chat ablak megjelenítését vezérli.
- isLoggedIn: A felhasználó hitelesített állapotát jelzi a localStorage alapján.
- userRole: A bejelentkezett felhasználó jogosultsági köre (user, admin vagy owner).
- unreadMessages: Az aktuális olvasatlan üzenetek száma (badge-ként jelenik meg az ikonon).

Tesztek

Backend

Összesen 157 teszt

- AdminRouter: 28 teszt
- Cart: 16 teszt
- Categories: 3 teszt
- Orders: 15 teszt
- productModel : 23 teszt
- products: 10 teszt
- review: 14 teszt
- user: 22
- userModel: 26

A tesztek 100%-a sikeres, a lefedettség kb 60%

Frontend

Összesen 24 teszt

- authentikáció: 7 teszt
- cart: 10 teszt
- checkout: 5 teszt
- productCard: 2 teszt

A tesztek 100%-a sikeres, a lefedettség kb 25%;

Továbbfejlesztési javaslatok

- kártyás fizetés hozzáadása
- kedvezmények hozzáadása
- közösségi fórum hozzáadása

Források

- https://www.descope.com/learn/post/jwt
  2026.04.20
- gemini AI
  2026.04.20
- github copilot
  2026.04.20
- https://www.udemy.com/course/nodejs-the-complete-guide/learn/lecture/12167300?start=0#overview
  2026.03.15
- https://www.udemy.com/course/react-the-complete-guide-incl-redux/learn/lecture/35734402?start=135#overview
  2026.02.20